

プログラミング言語 Egison 入門

江木 聡志

2020 年 3 月 1 日

はじめに

本書は、著者が自作のプログラミング言語 Egison を通して提唱しているパターンマッチ指向プログラミングという新しいプログラミング・パラダイムをできるだけコンパクトに紹介することを目的として執筆された。パターンマッチまわりの Egison の構文の紹介と、それらの構文を使ったプログラミングテクニックの紹介を軸として執筆した。

Egison はより直感的にアルゴリズムを表現したいという動機で、主に著者自身により発案されたプログラミング言語のアイデアを実証するためにつくられた proof of concept のプログラミング言語である。Egison に実装されているこれらのアイデアのうち、もっとも重要な機能は本書の主題であるパターンマッチである。Egison のパターンマッチの特徴は、ユーザーにより拡張可能でかつ、非線形パターン（パターン変数に束縛した値を同じパターン内で参照するパターン）をバックトラッキングにより効率的に処理することである。本書の目的は、この機能がいかに表現力が豊かで、応用範囲がひろく、将来性が高いのか読者に著者と同じくらいに感じてもらうことである。本書ではふれないが、他の重要な機能には、微分幾何の計算を表現するのに便利なテンソルの添字記法をプログラム中で使うための機能がある。

本書内のサンプルプログラムは、現在の版の Egison の文法を使って記述されている。以降の開発で Egison の文法が変わる可能性が高い。しかし、サンプルプログラムの裏のアイデアの根幹は普遍的なものであるので、安心して読んでほしい。

本書がきっかけとなって、プログラミング言語、とくにパターンマッチの研究をする研究仲間が読者の中から現れてくれたらとてもうれしい。

2020 年 1 月 6 日

江木 聡志

目次

はじめに	ii
第 1 章 プログラミング Egison とは	1
1 プログラミング言語全体の中での Egison の位置づけ	1
2 パターンマッチ指向プログラミングとは	2
3 本書の構成	3
第 2 章 Egison 早巡り	4
1 matchAll式によるパターンマッチ	4
2 値パターンと述語パターンによる非線形パターンの表現	5
3 バックトラッキングによる効率的な非線形パターンマッチ	6
4 マッチャーによるパターンの拡張性と多相性	7
5 matchAllDFS式によるパターンマッチ結果の順序の制御	7
6 and パターン・or パターン・not パターン	8
7 ループ・パターン	9
8 シーケンシャル・パターン	10
9 forall パターン	11
10 パターン関数によるパターンのモジュール化	12
11 マッチャー合成による新しいマッチャーの生成	12
第 3 章 パターンマッチ指向プログラミング入門	14
1 パターンマッチによるリストプログラミング	14
2 多重集合プログラミング	14
3 ネストしたループ・パターン	16
4 タプル・パターンによるデータの比較	16
第 4 章 より実践的なパターンマッチ指向プログラミング	17
1 SAT ソルバー	17
2 ゲーム木のパターンマッチ	19

第 5 章	マッチャーを定義しよう	20
1	Egison 内部のパターンマッチアルゴリズム	20
2	unorderedPairマッチャーの定義	21
3	multisetマッチャーの定義	21
4	soretedListマッチャーの定義	23
第 6 章	Egison パターンマッチの実装	25
1	インタプリタ実装	25
2	ほかの関数型言語へのトランスレーター	25
第 7 章	今後の展開	26
1	Egison への静的型システムの実装	26
2	定理証明システムへの Egison パターンマッチの導入	26
3	数式処理システムとしての機能の拡充	26
4	より効率の良いパターンマッチ・アルゴリズムの開発	26
5	パターンマッチ指向プログラミングの探求	26
付録 A	パターンマッチ指向プログラミング問題集	27
	あとがき	31
	参考文献	32
	索引	33

第 1 章

プログラミング Egison とは

本章では、Egison がどのような言語であるのかを紹介する。本章の内容は、いま完全に理解する必要はない。Egison とそれが提唱しているパターンマッチ指向プログラミングがどのような点で優れているのかその大体のイメージをつかんでもらいたい。

1 プログラミング言語全体の中での Egison の位置づけ

数多あるプログラミング言語のなかでの Egison の著しい特徴は、世界で初めて自身に実装されたアルゴリズムの記述を直感的にするための機能を有しているところである。アルゴリズムの記述を本質的に簡潔にするプログラミング言語のアイデアは、歴史を紐解いてみても、そう多くはない。そういった意味で Egison は数多にあるプログラミング言語のなかでも希少な言語であるだろう。

プログラミング言語の機能には大きく分けて 2 種類ある。コンピューターを含むさまざまな機械の制御を簡潔に記述するための機能と、数学的なアルゴリズムを簡潔に記述するための機能である。前者の機能は、たとえば、オペレーティング・システムやウェブ・ブラウザなど我々が日常的に使用しているさまざまなソフトウェアを実装する上で重要な機能である。後者の機能は、(TODO:)。Egison は後者の機能の拡充に注力してつくられている。

アルゴリズムを簡潔に記述することを目指しているプログラミング言語の流派の主流は、関数型プログラミング言語である。関数型プログラミングでは、コンピューターがプログラムを実行するのはプログラムはあまり意識せず、アルゴリズムを数学的にそのまま記述することを目指す。関数型プログラミングでは、コンピューターが実行できる形にプログラムを書き直す役割は主にコンパイラが果たす。

関数型プログラミング言語によるプログラムの記述の大きな特徴は、関数を他の組み込みデータ型と同じようにプログラマが扱えることである。具体的にいうと、関数を別の関数の引数として渡したり、関数を返す関数を定義することができる。そのおかげで、関数型プログラミング言語以外の言語ではモジュール化がむずかしい処理がモジュール化できることが多々ある。

しかし、関数型プログラミング言語でも、頭のなかのアルゴリズムのイメージを、関数型プログ

ラムとして記述できるように翻訳する必要がある場合もある。(TODO:)

2 パターンマッチ指向プログラミングとは

本節では、いくつかの例をみることによって、パターンマッチ指向プログラミングとはどのようなプログラミング・パラダイムであるのかそのイメージを紹介する。

パターンマッチ指向プログラミングによってプログラムの記述が直感的になっていることわかりやすい例に `intersect` 関数の実装がある。 `intersect` は 2 つのリストを引数にとり、それらのリストに共通する要素のリストを返す関数である。 `Egison` を使って引数のリストをそれぞれ多重集合としてパターンマッチすると、2 つのリストの共通要素にマッチするパターンを記述することにより、 `intersect` を記述できる。^{*1}

```
1 intersect xs ys :=
2   matchAllDFS (xs, ys) as (multiset eq, multiset eq) with
3   | ($x :: _, #x :: _) -> x
```

対して、 `Egison` のパターンマッチを使わずに関数型プログラミングのスタイルで `Haskell` で記述すると、リストに対する関数を組み合わせて共通要素を抜き出すための方法を記述する必要がある。

```
1 intersect xs ys = filter (\x -> any (== x) ys) xs
```

このプログラムは、リスト内包表記を使って以下のように書き直すこともできる。

```
1 intersect xs ys = [x | x <- xs, any (== x) ys]
```

`Egison` のパターンマッチを使ったプログラムは、2 つのリストの共通要素にマッチするパターンを記述しているだけであるのに対し、関数型プログラミングでは、どうやってその共通要素を取り出すかその方法をプログラマが考えて記述する。前者のように「何を計算したいか (what to do)」を記述するプログラミングスタイルは宣言型プログラミング、後者のように「どのように計算するか (how to do)」を記述するプログラミングスタイルは手続き型プログラミングと呼ばれる。 「何を計算したいか」から「どのように計算するか」が明らかである場合は、「何を計算したいか」を記述する宣言型プログラミングのほうがプログラムが読みやすく簡潔になる。 `intersect` の例の場合は、 `Egison` が多重集合のパターンマッチアルゴリズムをモジュール化できるおかげで、宣言的なプログラミングが可能になっている。

少し毛色の違う例として、 `concat` 関数をパターンマッチ指向プログラミングで定義する。リストのリストの要素にマッチするパターンを記述することにより、 `concat` を定義できる。

```
1 concat xss :=
2   matchAllDFS xss as list (list something) with
```

^{*1} プログラムの読み方は次章以降で紹介する。

```

3 | _ ++ (_ ++ $x :: _) :: _ -> x
4
5 concat [[1,2],[3],[4,5]]
6 -- [1,2,3,4,5]

```

さらにおもしろいパターンマッチ指向プログラミングの例として、unique関数を定義する。後方に自身と同じ値が含まれない要素にマッチするパターンを記述することにより uniqueを定義できる。

```

1 unique xs :=
2   matchAllDFS xs as list eq with
3   | _ ++ $x :: !(_ ++ #x :: _) -> x
4
5 unique [1,2,3,2,4]
6 -- [1,3,2,4]

```

次章以降で、上記のプログラムで使われている Egison の構文や、機能、プログラミングテクニックの解説をしていく。

3 本書の構成

本書は可能な限りコンパクトに、パターンマッチ指向プログラミングとそのための Egison の機能の解説をまとめることを目指した。そのため、本書は前提知識として、関数型プログラミング (Lisp 系言語や, OCaml, Haskell を使ったプログラミング) の知識を仮定している。既存の関数型言語にも存在する機能については、独立した節を設けず、登場したときに軽く解説するにとどめた。

本書の構成は以下の通りである。第 2 章では、パターンマッチに関係する Egison の構文を一通り紹介する。第 3 章では、Egison のパターンマッチを活かしたプログラミングスタイルであるパターンマッチ指向プログラミングとはなにか解説する。第 4 章では、より実践的なパターンマッチ指向プログラミングの例を紹介する。第 5 章では、ユーザーによるマッチャー定義の方法を説明する。また、それに伴って Egison 内部のパターンマッチの仕組みも解説する。第 6 章では、Egison のパターンマッチの実装を紹介する。第 7 章では、今後の研究課題について述べる。

ユーザーとして Egison に興味ある方は、第 2 章、第 3 章、第 4 章を順に読んでいくのがよい。Egison の設計に興味ある方は、第 2 章の前半部分を読んだ後、第 5 章を読むことも可能である。また、すべての章を読めば、Egison の研究もといパターンマッチ指向プログラミングの研究を開始することができるはずである。そのような読者が現れてくれたら、とてもうれしい。

第 2 章

Egison 早巡り

本章は、パターンマッチ指向プログラミングのための Egison の機能を一通り紹介する。

本章の後半で紹介されるループ・パターンや、シーケンシャル・パターン、forall パターンは Egison 以外のプログラミング言語ではみることがない独自のパターンである。

1 matchAll 式によるパターンマッチ

(TODO: 非自由データ型の説明)

パターンマッチを記述するためのいくつかの構文を Egison は提供している。そのうちもっとも基本的な構文は matchAll 式である。

```
1 matchAll [1,2,3] as list something with
2 | $x :: $ts -> (x, ts)
3 -- [(1,[2,3])]
```

matchAll 式は、ターゲット（上記の場合、[1,2,3]）、マッチャー（上記の場合、list something）、マッチ節（上記の場合、\$x :: \$xs -> (x,xs)）から構成される。マッチ節は、パターン（上記の場合、\$x :: \$xs）とボディ（上記の場合、(x,xs)）からなる。matchAll 式は、既存のプログラミング言語のマッチ式と同様に、ターゲットとパターンのパターンマッチを試みて、もしマッチしたらそのマッチ節のボディを評価する。

Egison の matchAll 式の特徴は、(1) matchAll という名前と結果としてリストを返すことと、(2) マッチャーという追加の引数をとることにある。(1) の特徴は、複数の結果をもつパターンマッチをサポートするための特徴である。matchAll 式は複数のパターンマッチの結果すべてについてボディを評価して、その結果を集めてリストとして返す。上記の例のパターンに使われている :: はと呼ばれるリストを先頭の要素と残りのリストに分解するパターンコンストラクタである。そのため、上記の例の場合は、パターンマッチの結果が一つであるので、長さが 1 のリストを返している。(2) の特徴は、パターンマッチアルゴリズムの拡張性とパターンの多相性を実現するための特徴である。マッチャーは Egison 以外の言語ではみられない独自のオブジェクトで、パターン

マッチアルゴリズムを保持するためのオブジェクトである。マッチャーによるパターンの多相性については、4 節で扱う。

--は Egison のインライン・コメント・デリミタである。本書を通して、プログラム直後のインライン・コメントを使って、プログラムの実行結果を記述する。

matchAll 式の文法の詳細についてももう少し説明する。as と with というキーワードでマッチャーは囲まれる。matchAll 式はマッチ節を複数とることができる。([TODO: 複数のマッチ節をとる matchAll の例へのリンク](#)) マッチ節同士の間は、| で区切られる。最初のマッチ節の前にも | は必須である。| はマッチ節が複数行に渡る場合にプログラムの可読性を高める。マッチ節中のパターンとボディは->で区切られる。

```
1 matchAll ターゲット as マッチャー with
2 | パターン -> ボディ
3 | パターン -> ボディ
4 ...
```

マッチ節が 1 つであるときは、下記のようにマッチ節の前の | を省略することができる。

```
1 matchAll ターゲット as マッチャー with パターン -> ボディ
```

```
1 matchAll [1,2,3] as list something with
2 | $hs ++ $ts -> (hs, ts)
3 -- [([], [1, 2, 3]), ([1], [2, 3]), ([1, 2], [3]), ([1, 2, 3], [])]
```

2 値パターンと述語パターンによる非線形パターンの表現

matchAll 式は、非線形パターンと組み合わせたときにその本領を発揮する。たとえば、以下の非線形パターンは、対象のコレクションが同値な要素のペアを含む場合にパターンマッチに成功する。

```
1 matchAll [1,2,3,2,4,3] as list integer with
2 | _ ++ $x :: _ ++ #x :: _ -> x
3 -- [2,3]
```

値パターンは非線形パターンを表現するために重要な役割を果たす。値パターンは、値パターンの中身とターゲットが等しいかどうかチェックする。値パターンの先頭には、# が付加される。# の後ろには任意の式を書くことができる。# に続く式は、その値パターンの左側に現れたパターン変数に束縛された値を参照しながら評価される。この動作を実現するため、Egison のパターンは左から右に順番に処理されることが決まっている。その結果、\$x :: #x :: _ のようなパターンは妥当であるが、#x :: \$x :: _ は正しく動かない。(どうしてもパターンの右側に現れるパターン変数の値を参照したいという場合は、8 節で紹介するシーケンシャル・パターンが使える。)

よりおもしろい非線形パターンの例として、双子素数のパターンマッチを紹介する。双子素数とは、 $(p, p+2)$ という形の素数のペアのことをいう。primesには素数の無限列が束縛されている。primesは Egison の組み込みライブラリで定義されている。この matchAll 式は、素数の無限列からすべての双子素数を順番に抜き出す。

```
1 twinPrimes := matchAll primes as list integer with
2   | _ ++ $p :: #(p + 2) :: _ -> (p, p + 2)
3
4 take 8 twinPrimes
5 -- [(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73)]
```

パターンマッチのときに、同値性よりも一般的な述語を使いたいことがある。述語パターンは、そのために用意されている組み込みパターンである。述語パターンは、述語パターンの中身の述語にターゲットを適用したときに真が帰ってきた場合、パターンマッチに成功するパターンである。述語パターンの先頭は、?からはじまり、?のあとには 1 引数の述語が続く。

```
1 twinPrimes = matchAll primes as list integer with
2   | _ ++ $p :: ?(\q -> q = p + 2) :: _ -> (p, p + 2)
```

3 バックトラッキングによる効率的な非線形パターンマッチ

Egison 内部のパターンマッチアルゴリズムは、非線形パターンを効率的に処理するためにバックトラッキングを使う。

```
1 matchAll [1..n] as list integer with _ ++ $x :: _ ++ #x :: _ -> x
2 -- 0(n^2) で [] を返す
3 matchAll [1..n] as list integer with _ ++ $x :: _ ++ #x :: _ ++ #x :: _ -> x
4 -- 0(n^2) で [] を返す
```

上記の 2 つのマッチ式は、1 から n までの整数のリストから同じ要素の 2 つ組、3 つ組をそれぞれ抽出する。同じ要素の 2 つ組も 3 つ組もターゲットのリストには含まれていないため、これらの matchAll 式は両方とも空リストを返す。

2 つ目の matchAll 式が評価されるとき、Egison 処理系は 2 つ目の #x のパターンマッチまで行き着かない。1 つ目の #x でパターンマッチが失敗するからである。(2 節で述べたように Egison のパターンマッチはパターンを左から右へ順番に処理する。) それゆえ、両方の matchAll 式の時間計算量は同じである。Egison 内部のパターンマッチアルゴリズムについては、第 5 章 1 節で詳しく解説する。

4 マッチャーによるパターンの拡張性と多相性

パターンの拡張性以外のマッチャーがもたらすメリットに、パターンのアドホック多相性がある。パターンのアドホック多相性は、さまざまな非自由データ型のパターンマッチを許す Egison において非常に重要である。なぜなら、ある同じデータがプログラムの複数の箇所ですれぞれ違う非自由データ型としてパターンマッチされることが多々あるためである。たとえば、コレクションはリストや多重集合、集合としてパターンマッチされる。パターンの多相性は、パターンコンストラクタの名前の数を大幅に減らす。

以下の `matchAll` 式は、コレクション `[1,2,3]` をターゲットとして、それぞれ違うマッチャーを使って同じコンスパターンでパターンマッチしている。マッチャーがリストの場合、コンスパターンは、先頭の要素と残りの要素のコレクションに分解する。マッチャーが多重集合の場合、コンスパターンは、ある要素と残りの要素のコレクションに分解する。マッチャーが集合の場合、コンスパターンは、ある要素とターゲットのコレクション自身に分解する。集合のこの挙動は、集合をすべての要素を無限個含むコレクションとしてとらえれば、自然な仕様と考えることができる。 $(\infty - 1 = \infty$ と考える.)

```
1 matchAll [1,2,3] as list something with $x :: $xs -> (x,xs)
2 -- [(1,[2,3])]
3 matchAll [1,2,3] as multiset something with $x :: $xs -> (x,xs)
4 -- [(1,[2,3]),(2,[1,3]),(3,[1,2])]
5 matchAll [1,2,3] as set something with $x :: $xs -> (x,xs)
6 -- [(1,[1,2,3]),(2,[1,2,3]),(3,[1,2,3])]
```

パターンの多相性は特に値パターンを表現するときに便利である。コンスパターンなどのコンストラクタパターンと同様に値パターンの挙動もマッチャーによって異なる。たとえば、`[1,2,3] == [2,1,3]` のようなコレクション同士の同値性は、コレクションをリストとしてみなすか多重集合としてみなすかによって変わってくる。しかし、Egison では、パターンのアドホック多相性のおかげで、同じ構文で両者の同値性をパターンでチェックできる。値パターンの多相性は、非自由データ型を扱うプログラムの読解のしやすさを大幅に向上させる。

```
1 matchAll [1,2,3] as list integer with #[2,1,3] -> "Matched" -- []
2 matchAll [1,2,3] as multiset integer with #[2,1,3] -> "Matched" -- ["Matched"]
```

5 `matchAllDFS` 式によるパターンマッチ結果の順序の制御

`matchAll` 式は、可算無限個の結果すべてを列挙するように内部のパターンマッチアルゴリズムが設計されている。そのため、パターンマッチの結果の順序にユーザーは気を付ける必要がある場合がある。

典型的な例を紹介する。以下の `matchAll` 式はすべての自然数のペアを列挙する。 `take` 関数を使って先頭 8 個のペアを取り出している。 `matchAll` はパターンマッチの探索木をすべてのノードをたどるために幅優先探索する。^{*1} その結果、パターンマッチの結果の順序は以下のようになる。

```
1 take 8 (matchAll [1..] as set something with $x :: $y :: _ -> (x,y))
2 -- [(1,1),(1,2),(2,1),(1,3),(2,2),(3,1),(2,3),(3,2)]
```

上記の順序は無限の大きい可能性がある探索木をすべてたどることには適している。しかし、この順序が好ましくない場面もある。たとえば、2 節で登場した `concat` や `unique` のパターンマッチはそうである。パターンマッチの探索木を深さ優先探索する `matchAllDFS` は、このような場面のために用意されている。

```
1 take 8 (matchAllDFS [1..] as set something with $x :: $y :: _ -> (x,y))
2 -- [(1,1),(1,2),(1,3),(1,4),(1,5),(1,6),(1,7),(1,8)]
```

6 and パターン・or パターン・not パターン

`and` パターンや `or` パターンをはじめとする論理パターンも、パターンの表現力を広げるために重要な役目を果たす。 `and` パターンと `or` パターンの使い方は、既存関数型言語とほとんど同じである。対して、`not` パターンは複数の結果をもつ非線形パターンマッチをサポートする Egison ならではのパターンである。

`and` パターンと `or` パターンの使用例として、三つ子素数を抽出するパターンマッチを紹介する。三つ子素数とは、 $(p, p+2, p+6)$ または $(p, p+4, p+6)$ の形の素数の三つ組のことをいう。 `and` パターンは、`as` パターンのように使われている。 `or` パターンは、 $p+2$ と $p+4$ の両方にマッチするために使われている。

```
1 primeTriples = matchAll primes as list integer with
2   | _ ++ $p :: ((#(p + 2) | #(p + 4)) & $m) :: #(p + 6) :: _
3   -> (p, m, p + 6)
4
5 take 6 primeTriples -- [(5,7,11),(7,11,13),(11,13,17),(13,17,19),(17,19,23),(37,41,43)]
```

`not` パターンは、その名前が示すとおり、ターゲットがパターンとマッチしない場合にパターンマッチに成功する。 `not` パターンの先頭には、`!` が付加される。 `!` の後に、任意のパターンが続く。以下の `matchAll` は双子素数でない隣り合う素数のペアを列挙する。

```
1 take 10 (matchAll primes as list integer with _ ++ $p :: (!#(p + 2) & $q) :: _ -> (p, q))
2 -- [(2,3),(7,11),(13,17),(19,23),(23,29),(31,37),(37,41),(43,47),(47,53),(53,59)]
```

^{*1} この幅優先探索の詳細については、Egison パターンマッチの設計についての論文 [2] の 5.2 節にて詳しく解説されている。

7 ループ・パターン

ループ・パターンはパターンの複数回の繰り返しを表現するための組み込みパターンである。ループ・パターンは正規表現のクリーネスター演算子 (*) の拡張である。

ターゲットのコレクションの 2 つの要素の組み合わせを列挙するパターンマッチを考えることから始める。これは以下のような `matchAll` 式で記述できる。

```
1 comb2 xs = matchAll xs as list something with _ ++ $x_1 : _ ++ $x_2 : _ -> [x_1, x_2]
2
3 comb2 [1,2,3,4] -- [[1,2],[1,3],[2,3],[1,4],[2,4],[3,4]]
```

Egison はこの例の中の `$x_1` と `$x_2` のように、パターン変数に添字を付加することを許す。これらは添字付き変数と呼ばれ、 x_1 や x_2 のような数式に対応する。_のあとに続く式は、添字と呼ばれ、整数に評価される必要がある。`xijk` のように任意個の添字を変数に付加することができる。添字付き変数 `$xi` に値が束縛されたとき、もし変数 `x` にまだ何も束縛されていなかった場合に、Egison 処理系は、連想配列を生成し、`x` に束縛する。この連想配列のキーは整数 `i` であり、それに対応する値は添字付き変数 `$xi` にマッチした値である。変数 `x` にすでに連想配列が束縛されている場合には、新しいキーとバリューのペアがこの連想配列に追加される。

上記の `comb2` の 2 を n に一般化してみよう。ここで、ループ・パターンを使うことができる。

```
1 comb n xs = matchAll xs as list something with
2     loop $i (1,n)
3         (_ ++ $x_i : ...)
4         _ -> map (\i -> x_i) [1..n]
5
6 comb 2 [1,2,3,4] -- [[1,2],[1,3],[2,3],[1,4],[2,4],[3,4]]
7 comb 3 [1,2,3,4] -- [[1,2,3],[1,2,4],[1,3,4],[2,3,4]]
```

ループ・パターンは、添字変数 (*index variable*)、添字範囲 (*index range*)、繰り返しパターン (*repeat pattern*)、終端パターン (*final pattern*) を引数にとる。添字変数は、現在の繰り返し回数を保持するための変数である。添字範囲は、添字変数が動く範囲を指定するために使われる。添字範囲は、開始値 (*initial number*) と終了値 (*final number*) のペアである。繰り返しパターンは、添字変数が添字範囲のなかにいる間、繰り返されるパターンである。終端パターンは、添字変数が添字範囲の外に動いたときに展開されるパターンである。

ループ・パターンの中では、三点リーダーパターン (*ellipsis pattern*) ... が使われる。繰り返しパターンや終端パターンは、三点リーダーパターンの場所に展開される。三点リーダーパターンで繰り返しパターン展開されるとき、添字変数の値がインクリメントされる。

上記のループ・パターンの繰り返し回数は定数だった。しかし、添字範囲の終了値を整数値でなくパターンにすることにより、ターゲットによりループ・パターンの繰り返し回数が変わるように

することができる。添字範囲の終了値がパターンである場合、三点リーダーパターンは繰り返しパターンと終端パターンの両方に展開される。三点リーダーパターンが終端パターンに展開されたときの繰り返し回数が、添字範囲の終了値のパターンとパターンマッチされる。以下のループ・パターンは、ターゲットのコレクションの先頭部分を列挙する。

```
1 matchAll [1,2,3,4] as list something with loop $i (1, $n) ($x_i : ...) _ -> map (\i ->
  x_i) [1..n]
2 -- [[],[1],[1,2],[1,2,3],[1,2,3,4]]
```

ループ・パターンは、ツリーやグラフのパターンマッチをするときに特によく使われる。たとえば、第4章2節のゲーム木のパターンマッチでも使われる。形式的なループ・パターンの構文と意味論の解説は、ループ・パターンの論文 [1] で紹介されている。

8 シーケンシャル・パターン

Egison 処理系はパターンを左から右に順番に処理する。しかし、パターンの右側で束縛されるパターン変数の値を参照したいなど、この処理の順番を変えたいことがある。シーケンシャル・パターンはそのために用意された組み込みパターンである。

シーケンシャル・パターンは、パターンマッチの処理の順番をユーザーが変えることを許す。シーケンシャル・パターンは、パターンのリストとして表現される。パターンマッチは、リストの先頭から順番に実行される。以下のシーケンシャルパターンは、リストの3つ目の要素、1つ目の要素、2つ目の要素の順番でターゲットのリストをパターンマッチする。

```
1 matchAll [2,3,1,4,5] as list integer with
2   [ @ :      @      : $x : _,
3     #(x + 1), @),
4     #(x + 2) -> "Matched" -- ["Matched"]
```

シーケンシャル・パターン中に現れる@は、後回しパターン変数 (*later pattern variable*) と呼ばれる。後回しパターン変数に束縛されたターゲットは、シーケンシャル・パターンの次の要素のパターンでパターンマッチされる。複数の後回しパターン変数が現れた場合、次のシーケンスはそれらをまとめてタプルとしてパターンマッチする。シーケンシャル・パターンのこの特徴は、パターンの離れた複数の部分についてまとめて not パターンを適用することを可能にする。

たとえば、以下のシーケンシャル・パターンは、ターゲットのコレクションのペアが、共通の要素をちょうど1つだけ含む場合に、パターンマッチに成功する。シーケンシャル・パターンは、それぞれのコレクションに共通の要素があることをチェックした後に、それぞれの残りの要素のコレクションにこれ以上共通要素がないことをチェックすることを可能にする。シーケンシャル・パターンと not パターンの組み合わせは、数学的なアルゴリズムを記述するときに現れることがある。たとえば、第4章1節でも現れる。

```
1 singleCommonElem = match (xs, ys) as (multiset eq, multiset eq) with
```

```

2 | [($x :: @, #x :: @),
3 | ($y :: _, #y :: _)] -> True
4 | _ -> False

```

シーケンシャル・パターンと同等のことが `matchAll` 式のネストにより表現できるのではと考えた読者がいるかもしれない。実は、少なくとも 2 つの理由でこれは不可能である。1 つ目の理由は、ネストした `matchAll` 式は、Egison 内部の幅優先探索を壊すことに由来する。外側の `matchAll` 式の 2 番目の結果は、外側の `matchAll` 式 1 つ目の結果について内側の `matchAll` 式の結果をすべて評価した後に計算される。2 つ目の理由は、後回しパターン変数がターゲットだけでなく、マッチャーの情報も保持することである。`matchAll` の引数のマッチャーが、関数の引数に由来するパラメータであることがある。それゆえ、内側の `matchAll` 式で使うべきマッチャーが構文的に導けないことがある。

9 forall パターン

forall パターンは、すべての場合にパターンマッチに成功するとき、パターンマッチに成功するパターンを表現する。*forall* パターンは引数を 2 つとる。第一引数のパターンでまずパターンマッチし、そのすべての結果について第二引数がパターンマッチに成功するとき、成功結果を返す。第一引数と第二引数のパターンの関係は、シーケンシャルパターンの要素同士の関係と似ている。第一引数で `@` の部分にマッチしたターゲットとマッチャーは、第二引数のパターンとのパターンマッチが試みられる。

以下の *forall* パターンは、ターゲットのコレクションの要素がすべて奇数である場合にパターンマッチに成功するパターンを表現している。第一引数のパターン中に現れる `@` はターゲットのコレクションの要素にマッチする。第二引数の述語パターンで奇数かどうかチェックしている。パターンマッチに成功した場合、すべてのパターンマッチ結果を返す。

```

1 matchAll [1,5,3] as multiset integer with
2 | forall ((@ & $x) :: _) ?odd? -> x
3 -- [1,5,3]

```

1 つでも第二引数のパターンのパターンマッチに失敗する第一引数のパターンマッチの結果がある場合、全体のパターンマッチに失敗する。

```

1 matchAll [1,4,3] as multiset integer with
2 | forall ((@ & $x) :: _) ?odd? -> x
3 -- []

```

not パターンでも *forall* パターンに近い意味のパターンを表現できる。*not* パターンの *forall* パターンに対する利点は、以下の二点である。

- (1) *forall* パターン中のパターン変数に値を束縛し、複数のパターンマッチの結果すべてを返す

ことができる (not パターンの中身ではパターン変数に値を束縛することができない).
(2) forall パターンを使ったほうがより直接的に意味を表現できる場合がある.

```
1 matchAll [1,5,3] as multiset integer with
2 | !(?!odd? :: _) -> "match"
3 -- ["match"]
```

10 パターン関数によるパターンのモジュール化

コンス・パターンやジョイン・パターンのようなコンストラクタ・パターンは、第5章で解説されるようにマッチャーで定義される. コンストラクタ・パターンを組み合わせ、新しいコンストラクタ・パターンをつくることもできる. そのために、パターンを引数にとってパターンを返すパターン関数を使う.

(TODO:)

```
1 twin := \pat1 pat2 => (~pat1 & $x) :: #x :: ~pat2
```

(TODO:)

```
1 match [1, 1, 2, 3] as list integer with
2 | twin $n $ns -> [n, ns]
3 -- [1, [2, 3]]
```

(TODO:)

11 マッチャー合成による新しいマッチャーの生成

マッチャーは合成可能で、マッチャーを組み合わせ、たとえば、多重集合のタプルのマッチャーや多重集合の多重集合のマッチャーを定義することができる. これにより、さまざまなデータ型に対するマッチャーを定義することができる.

タプルに対するマッチャーは、マッチャーのタプルにより表現される. タプル・パターンはそのようなマッチャーを使ったパターンマッチのときに使われる. たとえば、以下は、第1章2節でも登場した intersect関数の定義である. 2つの多重集合のタプルに対するマッチャーを使って、コレクションの共通要素をパターンマッチで取り出している.

```
1 intersect xs ys =
2   matchAll (xs,ys) as (multiset eq, multiset eq) with
3   | ($x :: _, #x :: _) -> x
```

上記で使われている eqマッチャーは、同値性が定義されたデータ型^{*2}のパターンマッチに使える

^{*2} 現在の Egison には静的な型はないが、Haskell でいうと Eq型クラスに属するデータ型

ユーザー定義マッチャーである。eqマッチャーに対して、値パターンが使われた場合、同値性のチェックによりパターンマッチがおこなわれる。

また、タプル・マッチャーと、マッチャーを引数にとってマッチャーを返す関数を組み合わせると、さまざまな非自由データ型に対するマッチャーが定義できる。たとえば、辺の集合として表現したグラフのマッチャーが定義できる。以下の定義は、グラフのノードの ID が整数として表現されていることを仮定している。

```
1 graph = multiset (integer, integer)
```

隣接リストにより表現したグラフのマッチャーも定義できる。隣接リストによるグラフは、整数と整数の多重集合のタプルの多重集合として定義される。ここでも整数によりノードの ID を表現している。

```
1 adjacencyGraph = multiset (integer, multiset integer)
```

代数的データ型に対するマッチャーも定義できる。代数的データ型に対するマッチャーを定義するための特別な構文が Egison には用意されている。たとえば、二分木に対するマッチャーは algebraicDataMatcher 式を使って以下のように定義できる。

```
1 algebraicDataMatcher binaryTree a = BLeaf a | BNode a (binaryTree a) (binaryTree a)
```

代数的データ型に対するマッチャーと非自由データ型に対するマッチャーを組み合わせることができる。たとえば、任意の数の子供をもつツリーで子供の順番を無視するマッチャーは以下のように定義できる。第 4 章 2 節でこのツリーに対するパターンマッチを紹介する。

```
1 algebraicDataMatcher tree a = Leaf a | Node a (multiset (tree a))
```

第 3 章

パターンマッチ指向プログラミング 入門

Egison パターンマッチを活用したプログラミングにおいて，頻出するパターンを紹介する．

1 パターンマッチによるリストプログラミング

2 多重集合プログラミング

2.1 ポーカーの役判定

多重集合に対する非線形パターンを記述できる Egison を使うとポーカーの役判定が簡潔に記述できる．それぞれのポーカーの役は直接 1 つのパターンとして記述できる．著者が Egison のマッチ式を設計をしたとき，ポーカーの役判定をするプログラムを簡潔に記述できることを必要条件として設計した．

```
1 poker cs :=
2   match cs as multiset card with
3   | card $s $n :: card #s #(n-1) :: card #s #(n-2) :: card #s #(n-3) :: card #s #(n-4) ::
4     _
5     -> "Straight flush"
6   | card _ $n :: card _ #n :: card _ #n :: card _ #n :: _ :: []
7     -> "Four of a kind"
8   | card _ $m :: card _ #m :: card _ #m :: card _ $n :: card _ #n :: []
9     -> "Full house"
10  | card $s _ :: card #s _ :: card #s _ :: card #s _ :: card #s _ :: []
11    -> "Flush"
12  | card _ $n :: card _ #(n-1) :: card _ #(n-2) :: card _ #(n-3) :: card _ #(n-4) :: []
13    -> "Straight"
14  | card _ $n :: card _ #n :: card _ #n :: _ :: _ :: []
15    -> "Three of a kind"
```

```

15 | card _ $m :: card _ #m :: card _ $n :: card _ #n :: _ :: []
16   -> "Two pair"
17 | card _ $n :: card _ #n :: _ :: _ :: _ :: []
18   -> "One pair"
19 | _ :: _ :: _ :: _ :: _ :: _ :: [] -> "Nothing"

```

上記の poker関数は以下のように動く.

```

1 poker [Card Spade 5, Card Spade 6, Card Spade 7, Card Spade 8, Card Spade 9]
2 -- "Straight flush"
3 poker [Card Spade 5, Card Diamond 5, Card Spade 7, Card Club 5, Card Heart 5]
4 -- "Four of a kind"
5 poker [Card Spade 5, Card Diamond 5, Card Spade 7, Card Club 5, Card Heart 7]
6 -- "Full house"
7 poker [Card Spade 5, Card Spade 6, Card Spade 7, Card Spade 13, Card Spade 9]
8 -- "Flush"
9 poker [Card Spade 5, Card Club 6, Card Spade 7, Card Spade 8, Card Spade 9]
10 -- "Straight"
11 poker [Card Spade 5, Card Diamond 5, Card Spade 7, Card Club 5, Card Heart 8]
12 -- "Three of a kind"
13 poker [Card Spade 5, Card Diamond 10, Card Spade 7, Card Club 5, Card Heart 10]
14 -- "Two pair"
15 poker [Card Spade 5, Card Diamond 10, Card Spade 7, Card Club 5, Card Heart 8]
16 -- "One pair"
17 poker [Card Spade 5, Card Spade 6, Card Spade 7, Card Spade 8, Card Diamond 11]
18 -- "Nothing"

```

カードのマッチャーは以下のように代数的データマッチャーとして定義される.

```

1 suit := algebraicDataMatcher
2   | spade
3   | heart
4   | club
5   | diamond
6
7 card := algebraicDataMatcher
8   | card suit (mod 13)

```

3 ネストしたループ・パターン

3.1 N クイーン問題

3.2 グラフのパターンマッチ

4 タプル・パターンによるデータの比較

第 4 章

より実践的なパターンマッチ指向プログラミング

パターンマッチ指向プログラミングがどのような場面で役に立つか現在わかっている範囲で議論する。

1 SAT ソルバー

To see the effect of pattern-match-oriented programming for implementing practical algorithms, we implement a SAT solver. A SAT solver determines whether a given propositional logic formula has an assignment for which the formula evaluates to true. Input formulae for SAT solvers are often in *conjunctive normal form*. A formula in conjunctive normal form is a conjunction of clauses, which are disjunctions of *literals*. A literal is a formula whose form is p or $\neg p$. For example, $(p \vee q) \wedge (\neg p \vee r) \wedge (\neg p \vee \neg r)$ is a formula in conjunctive normal form that has a solution; $p = \text{False}$, $q = \text{True}$, and $r = \text{True}$.

1.1 The Davis-Putnam Algorithm

In our implementation, propositional logic formulae in conjunctive normal form are represented as a collection of collections of literals. We can pattern-match them as a multiset of multisets of literals because both $\wedge$ and $\vee$ are commutative operators. Furthermore, we represent a literal as an integer. We represent positive and negative literals as positive and negative integers respectively: for example, p and $\neg p$ are represented as 1 and -1 , respectively. Therefore, the matcher for these formulae can be defined by simply composing matchers as `multiset (multiset integer)`.

The program below shows the main part of our implementation of the Davis-Putnam algorithm[3]. The `sat` function takes a list of propositional variables and a logical formula,

and returns True if there is a solution, otherwise returns False.

```

1 sat vars cnf = match (vars, cnf) as (multiset integer, multiset (multiset integer)) with
2   ( _ , [] ) -> True
3   ( _ , [] : _ ) -> False
4   ( _ , ($x : []) : _ ) -> -- 1-literal rule
5     sat (delete (abs x) vars) (assignTrue x cnf)
6   ($v : $vs, !((#(negate v): _ ) : _)) -> -- pure literal rule (positive)
7     sat vs (assignTrue v cnf)
8   ($v : $vs, !((#v: _ ) : _)) -> -- pure literal rule (negative)
9     sat vs (assignTrue (negate v) cnf)
10  ($v : $vs, _ ) -> -- otherwise
11    sat vs (deleteClausesWith [v, negate v] cnf ++ resolveOn v cnf)

```

The first match clause states that the input formula has a solution when it is empty. The second match clause states that there is no solution when clauses include an empty clause. The third match clause represents *1-literal rule*. When the input formula includes a clause with a single literal, we can assign that literal True at once. The fourth match clause states that if there is a propositional variable that appears only positively, we can set the value of this literal True and remove the propositional variable of x from the variable list and the clauses that include this literal from the formula. For example, $(p \vee q) \wedge (\neg p \vee r) \wedge (\neg p \vee \neg r)$ contains a propositional variable q only positively, so we can assign q True and remove the first clause from the next recursion. The fifth match clause states that the opposite of the fourth match clause. It removes the clauses that include this literal if there is a propositional variable that appears only negatively (p in the above sample is such propositional variable). The final match clause applies the resolution principle. This match clause lists all pairs of clauses $p \vee C$ and $\neg p \vee D$ (let C and D be a disjunction of literals), and obtains new clauses $C \vee D$.

The above definition of `sat` describes all rules for simplifying an input formula by fully utilizing pattern matching for multisets. In traditional functional languages, we need to call several library functions and define several helper functions to describe these conditional branches. We can compare this implementation with the same algorithm implemented in OCaml in [3].

1.2 Pattern Matching for Resolution

We can elegantly define the `resolveOn` function using a sequential not-pattern.

First, let us show a naive implementation of `resolveOn`. The `resolveOn` function is defined with a single `matchAll` expression as follows.

```

1 resolveOn v cnf = matchAll cnf as multiset (multiset integer) with

```

```
2  (#v : $xs) : (#(negate v) : $ys) : _ -> unique (filter (\c -> not (tautology c)) (xs ++
    ys))
```

The pattern for enumerating the pair of clauses $p \vee C$ and $\neg p \vee D$ is described simply utilizing pattern-matching for a multiset of multisets. Note that the above `resolveOn` removes tautological clauses by using the `tautology` predicate in the body of the match clause.

We can remove this use of the `tautology` predicate by using a sequential not-pattern discussed in Sect. ?? . The sequential pattern is effectively used to describe that the literal `x` appeared in `xs` does not appear negatively in `ys`.

```
1 resolveOn v cnf = matchAll cnf as multiset (multiset integer) with
2   [(#v : (and @ $xs)) : (#(negate v) : (and @ $ys)) : _,
3    !($x : _, #(negate x) : _)] -> unique (xs ++ ys)
```

1.3 Separating Two Kinds of Loops

The SAT solver presented in this section is an important sample in the sense that it is the only sample that contains a loop that we cannot remove by pattern-match-oriented programming. This unremovable loop is the recursion of the `sat` function. This recursion is essential for narrowing the search tree. This narrowing is impossible by simple backtracking. On the other hand, all the other loops that can be implemented in backtracking algorithms are pushed into the patterns. In the traditional style, we describe the 1-literal rule and pure literal rules by using recursive functions such as `find`, `partition`, `subtract` and `intersect` [3]. Thus, the separation of two kinds of loops increases the readability of practical algorithms.

2 ゲーム木のパターンマッチ

第 5 章

マッチャーを定義しよう

1 Egison 内部のパターンマッチアルゴリズム

This section explains and implements the pattern-matching algorithm of Egison, which is described in detail in Sect. 5 and Sect. 7 of the original paper of Egison [2]. After the explanation of the pattern-matching algorithm, I introduce a simplified implementation of this algorithm in Scheme. This implementation is called by the proposed macros whose implementation is introduced in Sect. ??.

First, I start by explaining the pattern-matching algorithm of Egison briefly. In Egison, pattern matching is implemented as reductions of *matching states*. A matching state consists of a stack of *matching atoms* and an intermediate result of pattern matching. A matching atom is a triple of a pattern, target, and matcher. In each step of a pattern-matching process, the top matching atom of the stack of matching atoms is popped off. From this matching atom, a list of lists of next matching atoms is calculated. Each list of the next matching atoms is pushed on the stack of matching atoms of the matching state. As a result, a single matching state is reduced to multiple stacks of matching states in a single reduction step. Pattern matching is recursively executed for each matching state. When a stack becomes empty, it means pattern matching for this matching state succeeded. Fig. 5.1 shows the reduction path when executing the `match-all` expression below. The matching state that is not grayed-out in the i -th row is reduced to matching states in the $(i + 1)$ -th row. For example, the first matching state in the second row is reduced to the matching state in the third row. `MState` takes two arguments, a stack of matching atoms and an intermediate pattern-matching result. The value `pattern ,m` is transformed to `(val ,(lambda (m) m))`. This transformation is explained in Sect. ??.

```
1 (match-all '(2 8 2) (Multiset Integer) [(cons m (cons ,m _)) m]) ; (2 2)
```


1	(MState {[cons m (cons (val ,(lambda (m) m)) _)] (Multiset Integer) {2 8 2}} {})
	(MState {[m Integer 2] [(cons (val ,(lambda (m) m)) _)] (Multiset Integer) {8 2}} {})
2	(MState {[m Integer 8] [(cons (val ,(lambda (m) m)) _)] (Multiset Integer) {2 2}} {})
	(MState {[m Integer 2] [(cons (val ,(lambda (m) m)) _)] (Multiset Integer) {2 8}} {})
3	(MState {[m Something 2] [(cons (val ,(lambda (m) m)) _)] (Multiset Integer) {8 2}} {})
4	(MState {[cons (val ,(lambda (m) m)) _] (Multiset Integer) {8 2}} {2})
5	(MState {[m Integer 8] [_] (Multiset Integer) {2}} {2})
	(MState {[m Integer 2] [_] (Multiset Integer) {8}} {2})
6	(MState {[_] (Multiset Integer) {8}} {2})
7	(MState {[_ Something {8}}] {2})
8	(MState {} {2})

図 5.1 Reduction path of matching states

2 unorderedPair マッチャーの定義

3 multiset マッチャーの定義

Matcher definition is the most technical part in pattern-match-oriented programming. This section explains the method for defining a matcher by showing a matcher definition for multisets. The program below shows a matcher definition for multisets. The multiset matcher is the most basic nontrivial matcher. This section explores the detail of this definition.

```

1 multiset a = matcher
2     [] as () with
3         \tgt -> case tgt of
4             [] -> [()]
5             _ -> []
6     $ : $ as (a, multiset a) with
7         \tgt -> matchAll tgt as list a with
8             $hs ++ $x : $ts -> (x, hs ++ ts)
9     # $val as () with
10        \tgt -> match (val, tgt) as (list a, multiset a) with
11            ([], []) -> [()]
12            ($x : $xs, #x : #xs) -> [()]
13            (_, _) -> []
14    $ as something with
15        \tgt -> [tgt]
```

A matcher is defined using the `matcher` expression. The `matcher` expression is a built-in syntax of Egison. `matcher` takes a collection of *matcher clauses*. A *matcher clause* is a triple of a *primitive-pattern pattern*, a *next-matcher expression*, and a *next-target expression*.

A matcher is a kind of function that takes a pattern and target, and returns lists of the next *matching atoms*. A matching atom is a triple of a pattern, target, and matcher. A primitive-pattern pattern matches a pattern. Patterns that match with *pattern holes* (\$) inside primitive-pattern patterns) are next patterns. A next-matcher expression returns next matchers. A next-target expression is a function that takes a target and returns a list of next targets. A matcher generates a list of the next matching atoms by combining next patterns, next matchers, and a list of next targets.^{*1} p The *multiset* matcher has four matcher clauses. The first matcher clause handles a nil pattern, and it checks whether the target is an empty collection or not. The second matcher clause handles a cons pattern. The third matcher clause handles a value pattern. This matcher clause defines the equality of multisets. The fourth matcher clause handles the other patterns for multiset: a pattern variable and wildcard.

First, we focus on the second matcher clause. The primitive-pattern pattern of the second matcher clause is \$: \$, and the next matcher expression is (a, multiset a). It means two arguments of the cons pattern are next patterns and they are pattern-matched using the a and multiset a matchers, respectively. a is an argument of multiset and the matcher for inner elements of a multiset. In the next target expression, a simple join-cons pattern is used to decompose a target collection into an element and the rest collection. For example, when the target is a collection [1,2,3], this next-target expression returns [(1,[2,3]), (2,[1,3]), (3,[1,2])]. Each tuple of the next targets is pattern-matched using the next patterns and the next matchers recursively. For example, 1 and [2,3] are pattern-matched using the a and multiset a matcher with the first and the second argument of the cons pattern, respectively.

Next, we focus on the third matcher clause. This matcher clause is as technical as the second matcher clause. The primitive-pattern pattern of this matcher clause is #val. It is called a *value-pattern pattern*. A value-pattern pattern matches a value pattern. This matcher clause compares the content of a value pattern (val) and a target (tgt) as multisets. The match expression is used for this comparison. Interestingly, tgt is recursively pattern-matched as multiset a.

The first and the third match clauses of this match expression are simple. The first match clause states that it returns [] when both val and tgt are empty. This return value means pattern matching for the value pattern succeeded. The third match clause states that it returns [] if pattern matching for the patterns of both the first and the second match clause

^{*1} In Egison, pattern matching is implemented as reductions of stacks of matching atoms. Each list of the next matching atoms returned by a matcher is pushed to the stack of matching atoms. As a result, a single stack of matching atom is reduced to multiple stacks of matching atoms in a single reduction step. Pattern matching is recursively executed for each stack of matching atoms. When a stack becomes empty, it means pattern matching for this stack succeeded.

failed. This return value means pattern matching for the value pattern failed.

The second match clause is the most technical part of this match expression. The value pattern `#xs` is recursively pattern-matched using this matcher clause itself. The collection `xs` is one element shorter than `tgt`. Therefore, this recursion finally reaches the first or the third match clause if `val` and `tgt` are finite.

Finally, let us also explain the fourth matcher clause. This matcher clause creates the next matching atom by just changing the matcher from `multiset a` to `something`.

4 sortedList マッチャーの定義

Modularization of pattern-matching algorithms by matchers not by patterns enables polymorphic patterns. However, its merit extends beyond polymorphic patterns; matchers enable descriptions of more efficient pattern matching keeping patterns concise. The reason is that pattern matching against patterns inside matcher definitions allows us to describe more detailed pattern-matching algorithms. This section shows such an example, a matcher for sorted lists.

The program that used a doubly-nested join-cons pattern for enumerating pairs of prime numbers whose forms are $(p, p + 6)$ gets slower when the number of the enumerating prime pairs gets larger. The reason is that the program enumerates all the combinations of prime numbers. For example, the program tries to match all the pairs such as $(3, 5)$, $(3, 7)$, $(3, 11)$, $(3, 13)$, $(3, 17)$, $(3, 19)$, and so on. However, we should avoid enumerating the pairs after $(3, 11)$, the first pair whose difference is more than 6. This is because it is obvious that the difference between all the pairs after $(3, 11)$ are more than 6.

```
1 take 10 (matchAll primes as sortedList integer with
2     _ ++ #p : (_ ++ #(p + 6) : _) -> (p, p + 6))
3 -- [(5,11), (7,13), (11,17), (13,19), (17,23), (23,29), (31,37), (37,43), (41,47), (47,53)]
```

We can avoid this unnecessary search by creating a new matcher that is specialized for sorted lists. We can define such a matcher by adding a matcher clause with the primitive-pattern pattern `$ ++ #px : $` to the list matcher as shown below. This matcher clause improves the theoretical time complexity of the above pattern from $O(n^2)$ to $O(n)$.

```
1 sortedList a =
2   matcher
3     $ ++ #px : $ as (sortedList a, sortedList a)
4     \tgt -> matchAll tgt as list a with
5         loop $i (1, $n) ((and ?(\x -> x < px) $h_i) : ...) (#px : $ts) ->
6             (map (\i -> h_i) [1..n], ts)
7     ...
```

Note that this method is only applicable to Egison that modularizes pattern-matching methods for each matcher, not for each pattern. The reason is that we need to match patterns whose form is `_ ++ #x : _` as mentioned above. If pattern-matching methods are modularized for each pattern, we need to introduce a new pattern constructor `joinCons ... ..` that is equivalent to `_ ++ ... : ...` for this purpose.

第 6 章

Egison パターンマッチの実装

本章は、Egison パターンマッチの実装を紹介する。Egison パターンマッチの実装については、現在 2 本の論文と複数の実装が公開されている。本章はこれらの参考になる外部資料を紹介していきたい。

1 インタプリタ実装

河田旺さんによる Coq によるインタプリタ実装もある。([TODO: cite: formalized-egison](#))

2 ほかの関数型言語へのトランスレーター

Scheme マクロとしての実装

Haskell メタプログラミングによる実装

ループ・パターンやシーケンシャル・パターン、forall パターンといった Egison 独自のパターンで実装されていないパターンがある。その理由に添字付き変数など、どのように既存の言語の上に実装するのが適切なのか考えるのがむずかしいプログラミング言語の要素を含んでいるためである。

第 7 章

今後の展開

- 1 Egison への静的型システムの実装
- 2 定理証明システムへの Egison パターンマッチの導入
- 3 数式処理システムとしての機能の拡充
- 4 より効率の良いパターンマッチ・アルゴリズムの開発
- 5 パターンマッチ指向プログラミングの探求

付録 A

パターンマッチ指向プログラミング問題集

member?関数

下記の空白をうめて member?関数を完成させよ.

```
1 member? x xs :=
2   match xs as list eq with
3   |  -> True
4   | _ -> False
5
6 member? 1 [1,2,3,4]
7 -- True
8
9 member? 5 [1,2,3,4]
10 -- False
```

concat関数

下記の空白をうめて concat関数を完成させよ.

```
1 concat xss :=
2   matchAllDFS xss as  with
3   |  -> 
4
5 concat [[1,2],[3],[4,5]]
6 -- [1,2,3,4,5]
```

四つ子素数

四つ子素数を列挙するプログラムを記述せよ。四つ子素数とは、 $(p, p+2, p+6, p+8)$ という形の素数の四つ組のことをいう。

```
1 take 4 (matchAll primes as list integer with
2   |  -> (p, p + 2, p + 6, p + 8))
3 -- [(5, 7, 11, 13), (11, 13, 17, 19), (101, 103, 107, 109), (191, 193, 197, 199)]
```

差が6の素数のペア

下記の空白をうめて素数のペア $(p, p+6)$ を抽出せよ。

```
1 take 6 (matchAll primes as list integer with
2   |  -> (p, p + 6))
3 -- [(5, 11), (7, 13), (11, 17), (13, 19), (17, 23), (23, 29)]
```

intersect関数

下記の空白をうめて、2つのコレクションの共通部分を抽出する intersect関数を完成させよ。

```
1 intersect xs ys :=
2   matchAllDFS (xs, ys) as  with
3   |  -> 
4
5 intersect [1,2,3,4] [2,4,5]
6 -- [2,4]
```

difference関数

下記の空白をうめて、第二引数のコレクションに含まれていない第一引数のコレクションの要素のみを抽出する difference関数を完成させよ。

```
1 difference xs ys :=
2   matchAllDFS (xs, ys) as  with
3   |  -> 
4
5 difference [1,2,3,4] [2,4,5]
6 -- [1,3]
```


doubles関数

コレクション中にちょうど2回現れる要素のみを抽出する doubles関数を完成させよ。

```
1 doubles xs :=
2   matchAllDFS xs as  with
3   |  -> 
4
5 doubles [1,2,3,2,1,1,4,5,3]
6 -- [2,3]
```

tails関数

tails関数をパターンマッチを使って定義せよ。ループ・パターンを使う回答とそうでない回答の2つを考えてみよ。

```
1 tails xs :=
2   matchAll xs as list something with
3   | 
4   -> ts
5
6 tails [1,2,3,4]
7 -- [[1, 2, 3, 4], [2, 3, 4], [3, 4], [4], []]
```

多重集合の同値性チェック

多重集合の同値性をチェックする2引数述語 equalMultisetをパターンマッチを使って定義してみよ。

```
1 equalMultiset xs ys :=
2   match (xs, ys) as (list eq, multiset eq) with
3   | ([], [])
4   -> True
5   | 
6   -> equalMultiset xs' ys'
7   | _
8   -> False
9
10 equalMultiset [1,1,2] [2,1,1]
11 -- True
12
13 equalMultiset [1,1,2] [1,2,2]
```

```
14 -- False
```

ループ・パターンを使って再帰関数を使わずに書いてみよ.

```
1 equalMultiset xs ys :=
2   match (xs, ys) as (list eq, multiset eq) with
3   | 
4   -> True
5   | _
6   -> False
```

ジョーカーの存在を考慮するポーカーの役判定

3章 2.1 節のポーカーの役判定のためのパターンマッチ (poker関数) の実装を一切変更せずに, cardマッチャーの定義だけを変更して, ジョーカーの存在を考慮するポーカーの役判定をできるようにせよ.

```
1 card := matcher
2   | card $ $ as (suit, mod 13) with
3     | Card $s $n -> [(s, n)]
4     | Joker -> 
5   | $ as something with
6     | $tgt -> [tgt]
7
8 poker [Card Spade 5, Card Spade 6, Joker, Card Spade 8, Card Spade 9]
9 -- "Straight flush"
10 poker [Card Spade 5, Card Diamond 5, Joker, Card Club 5, Card Heart 7]
11 -- "Four of a kind"
```

あとかき

2020 年 2 月 (TODO:) 日 江木 聡志

参考文献

- [1] S. Egi. Loop Patterns: Extension of Kleene Star Operator for More Expressive Pattern Matching against Arbitrary Data Structures. In *The Scheme and Functional Programming Workshop*, 2018.
- [2] S. Egi and Y. Nishiwaki. Non-linear Pattern Matching with Backtracking for Non-free Data Types. In *Asian Symposium on Programming Languages and Systems*, pages 3–23. Springer, 2018.
- [3] J. Harrison. *Handbook of Practical Logic and Automated Reasoning*. Cambridge University Press, 2009.

索引

#, 5
--, 5
::, 4
?, 6

as, 5

forall パターン, 11

matchAll, 4
matchAllDFS, 8

primes, 6

with, 5

値パターン, 5

コンスパターン, 4
シーケンシャル・パターン, 10
述語パターン, 6
添字付き変数, 9
タプル・パターン, 12
パターンマッチ指向, 2
マッチャー, 4
ループ・パターン, 9